

SIMATS ENGINEERING

ASSESSMENT TOOL 2

K. Lokesh Chandra
192425358

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5

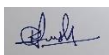
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I K. Lokesh Chandra (192425358) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

		solve the problem			
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. E-Commerce Product Records

Question: An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Answer

For 10 million product records, a **B+ Tree-based indexed file organization** is suitable.

Proposed Design

- Store product records in disk blocks.
- Use **ProductID as the primary key**.
- Maintain a **B+ Tree index** on ProductID.
- Use additional indexes on frequently searched fields such as CategoryID and Brand.
- Keep frequently accessed product details in optimized storage.

Why B+ Tree?

- Provides fast search with **$O(\log n)$** complexity.
- Supports efficient insertion and deletion.
- Leaf nodes are linked, making range queries efficient.
- Suitable for very large disk-based databases.
- Reduces the number of disk I/O operations.

Cost Analysis

Assume each record occupies approximately **200 bytes**.

Total storage = $10,000,000 \times 200$
= 2,000,000,000 bytes
 ≈ 1.86 GB

Using a 4 KB block:

Records per block = $4096 / 200$
 ≈ 20 records

Approximately:

Number of blocks = $10,000,000 / 20$
 $\approx 500,000$ blocks

A B+ Tree avoids scanning all 500,000 blocks for every product search.

Conclusion

A **B+ Tree indexed file organization** provides a good balance between storage cost, search speed, and scalability for 10 million products.

2. Banking Transaction Indexing

Question: A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy.

Answer

A **composite B+ Tree index** should be created on:

```
CREATE INDEX idx_account_date  
ON Transactions(AccountNumber, TransactionDate);
```

Design

Requirement	Index
Search by AccountNumber	B+ Tree
Search by AccountNumber + Date	Composite B+ Tree
Date range	B+ Tree
TransactionID lookup	Primary index

Example Query

```
SELECT *  
FROM Transactions  
WHERE AccountNumber = 10025  
AND TransactionDate BETWEEN '2026-01-01' AND '2026-01-31';
```

The index first locates the account number and then efficiently scans the required date range.

Advantages

- Fast account-based retrieval.
- Efficient date-range queries.
- Reduces disk I/O.
- B+ Tree supports ordered traversal.
- Suitable for large transaction tables.

Conclusion

The **composite (AccountNumber, TransactionDate) B+ Tree index** is the most appropriate choice because it supports both major access patterns efficiently.

3. Healthcare Patient Records

Question: A healthcare system stores patient records accessed by PatientID and doctor name. Design a multi-level indexing solution.

Answer

A **multi-level indexing strategy** can be designed using a primary index and secondary index.

Level 1 – Primary Index

Create a B+ Tree on:

PatientID

PatientID is unique and provides direct access to individual patient records.

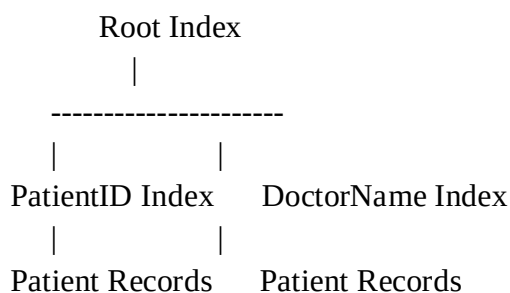
Level 2 – Secondary Index

Create a secondary B+ Tree on:

DoctorName

This allows all patients associated with a particular doctor to be found efficiently.

Structure



Example

```
CREATE INDEX idx_patient
ON Patient(PatientID);
```

```
CREATE INDEX idx_doctor
ON Patient(DoctorName);
```

Advantages

- PatientID search → **$O(\log n)$**
- Doctor name search → **$O(\log n)$**
- Supports multiple access paths.
- Reduces full-table scanning.
- Suitable for large healthcare databases.

Conclusion

Using **B+ Tree primary and secondary indexes** provides efficient retrieval through both PatientID and DoctorName.

Q4. B-Tree vs B+ Tree for Logistics

Question: A logistics company needs efficient dynamic insertion and deletion of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Answer

For a logistics system, a **B+ Tree is more suitable** than a B-Tree.

Feature	B-Tree	B+ Tree
Search	Fast	Fast
Insert	Efficient	Efficient
Delete	Efficient	Efficient
Range queries	Moderate	Excellent
Leaf linking	No	Yes
Disk access	Good	Better
Database usage	Good	Excellent

Reason

Shipment systems commonly require:

Find shipment by ID

Find shipments between two dates

Find shipments in a location range

Find recent shipments

B+ Tree leaf nodes are linked:

$[10,20] \rightarrow [25,30] \rightarrow [35,40] \rightarrow [45,50]$

Therefore, range queries can move sequentially between leaf nodes.

Complexity

Search = $O(\log n)$

Insert = $O(\log n)$

Delete = $O(\log n)$

Range = $O(\log n + k)$

where k is the number of matching records.

Conclusion

B+ Tree is the preferred solution because it provides efficient dynamic updates and excellent range-query performance.

5. HR System – Static vs Extendible Hashing

Question: Design a hashing scheme for an HR system with 5000 employees and evaluate static hashing versus extendible hashing.

Answer

For 5000 employees, use **EmployeeID as the hash key**.

Hash function:

$h(\text{EmployeeID}) = \text{EmployeeID} \text{ MOD } M$

For example, choose:

$M = 1024$

This distributes employees across 1024 buckets.

Static Hashing

EmployeeID $\rightarrow$ Hash Function $\rightarrow$ Fixed Bucket

Advantages:

- Simple implementation.
- Fast average search.
- Low management overhead.

Disadvantages:

- Number of buckets is fixed.
- Overflow chains may increase as employee data grows.
- Resizing is difficult.

Extendible Hashing

Extendible hashing uses:

Directory



Buckets

When a bucket overflows, it can split. If necessary, the directory doubles.

Comparison

Feature	Static Hashing	Extendible Hashing
Bucket size	Fixed	Dynamic
Overflow	Chaining	Bucket splitting
Growth	Difficult	Easy
Search	$O(1)$ average	$O(1)$ average
Scalability	Moderate	High
Implementation	Simple	More complex

Recommendation

For an HR system that may grow beyond 5000 employees, **extendible hashing is preferred**. It handles dynamic growth without requiring complete reorganization.

6. Airline Reservation System

Question: Support point queries by flight number and range queries by date. Propose a combined index structure.

Answer

A **combined indexing solution using B+ Tree indexes** is suitable.

Index 1 – Flight Number

```
CREATE INDEX idx_flight
```

```
ON Flights(FlightNumber);
```

This supports point queries:

```
SELECT *
```

```
FROM Flights
```

WHERE FlightNumber = 'AI202';

Index 2 – Date

CREATE INDEX idx_date

ON Flights(FlightDate);

This supports:

SELECT *

FROM Flights

WHERE FlightDate BETWEEN '2026-08-01'

AND '2026-08-15';

Structure

FlightNumber B+ Tree



Flight Records

FlightDate B+ Tree



Flight Records

Why B+ Tree?

- Excellent point search.
- Excellent range search.
- Sorted leaf nodes.
- Efficient disk-based retrieval.
- Supports large reservation databases.

Conclusion

Two B+ Tree indexes provide separate optimized access paths for **flight-number point queries** and **date-range queries**.

7. University Student Database

Question: Design file organization and secondary indexing for queries by department and CGPA range.

Answer

A **sorted file organization based on StudentID with secondary B+ Tree indexes** is suitable.

File Organization

StudentID → Sorted Student Records

Secondary Index 1

Department

CREATE INDEX idx_department

ON Student(Department);

Secondary Index 2

CGPA

CREATE INDEX idx_cgpa

ON Student(CGPA);

Example Queries

Department query:

```
SELECT *
```

```
FROM Student
```

```
WHERE Department = 'CSE';
```

CGPA range:

```
SELECT *
```

```
FROM Student
```

```
WHERE CGPA BETWEEN 8.0 AND 9.0;
```

Advantages

- Fast department-based retrieval.
- Efficient CGPA range search.
- B+ Tree supports ordered CGPA values.
- Reduces full-table scanning.
- Suitable for large student databases.

Conclusion

A sorted primary file with secondary B+ Tree indexes on Department and CGPA provides efficient access for both query types.

8. Disk Block Utilization

Question: Analyze disk block utilization for Heap, Sorted, and Hashed organizations using 100,000 records, 50 bytes each, and 4 KB blocks.

Given

Number of records = 100,000

Record size = 50 bytes

Block size = 4 KB = 4096 bytes

Step 1 – Records Per Block

$$4096 / 50 = 81.92$$

Therefore:

Records per block = 81

Step 2 – Number of Blocks

$$100,000 / 81 = 1234.57$$

Therefore:

Blocks required = 1235

Step 3 – Storage Used

$$1235 \times 4096$$

$$= 5,058,560 \text{ bytes}$$

$$\approx 4.82 \text{ MB}$$

Comparison

Organization	Approx. Blocks	Utilization	Main Advantage
Heap	1235	High	Fast insertion
Sorted	1235	High	Binary/range search



Scalability

For 500 million records:

- Use B+ Tree indexes for ordered queries.
- Partition data by time period.
- Consider hash partitioning by user_id.
- Use index maintenance carefully because indexes increase storage and write cost.

Conclusion

The combination of **(user_id, timestamp)** and **(hashtag, timestamp) indexes**, together with partitioning, provides scalable access to a very large social-media dataset.

10. Supermarket Billing System

Question: Compare insert, delete, and search performance of Heap, Sorted, and Hashed files for 1 million daily transactions.

Answer

For a supermarket billing system with 1 million daily transactions, the three organizations have different strengths.

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	O(1) average	O(n)	O(1) average
Delete	O(n)	O(n)	O(1) average
Search	O(n)	O(log n)	O(1) average
Range Search	Poor	Excellent	Poor
Storage	Low overhead	Moderate	Moderate
Best use	High insert workload	Ordered queries	Exact-key queries

Heap File

Records are inserted wherever free space is available.

Best for:

- High-volume transaction insertion.
- Sequential processing.

Problem:

Searching a transaction may require scanning many records.

Sorted File

Records remain ordered by transaction ID/date.

Advantages:

- Binary search.
- Efficient range queries.
- Ordered reporting.

Disadvantage:

- Insertions and deletions can be expensive because the sorted order must be maintained.

Hashed File

Use:

$h(\text{TransactionID}) = \text{TransactionID} \text{ MOD } M$

Advantages:

- Very fast exact-match retrieval.
- Fast average insertion.
- Fast average deletion.

Disadvantage:

- Collision handling is required.
- Poor for range queries.

Recommendation

For a supermarket billing system:

Primary transaction lookup → Hashing

Reporting/date ranges → B+ Tree

Temporary/high-volume data → Heap

A **hybrid solution** is therefore the most practical industry design.

Final Analysis

If the main operation is searching a transaction by its unique transaction ID, **hashed organization** provides the best average performance. If the system frequently generates date-based sales reports, a **B+ Tree index** should also be maintained.